RISC-V Single Cycle RV32I Processor

Project report by Julis Joy (24EC01023) and Nikhilesh Kurapati (24EC01026)

Table of Contents

Objective

To design and implement a single cycle RV32I processor on AMD's Basys 3 FPGA board using verilogHDL whilst ensuring a seamless datapath from instruction fetch to write back through decode, execute and memory access stages.

Key Features of this implementation

- Adjustable Clock Speed between 1Hz, 10Hz, 100Hz and 50 MHz, with hold switch.
- Ability to view the values of any one of the 32 registers on the on-board LEDs.
- Display of the current memory location on the on-board seven segment display.
- Multiple programs, with the ability to switch between them on the board itself.

Overview

RISC-V's ISA (Instruction Set Architecture) is a set of 32-bit machine-level instructions that defines how the CPU executes various operations such as arithmetic, load, store, branching and so on. RV32I's ISA comprises 40 machine-level instructions used to carry out these operations.These instructions are classified into R, I, S, U, B and J-type based on their function with each type having a unique format. The implemented processor supports 37 of the 40 instructions, excluding FENCE,ECALL and EBREAK and is single cycled, i.e., each instruction is executed in one clock cycle.

Processor Organisation

The processor has been divided into several modules, of which six are essential for the functioning, while two are feature additions. These are:

Critical Modules

1. Execution (Top Module)
2. ALU (Arithmetic Logic Unit)
3. Data Memory
4. Instruction Memory
5. Instruction Decode
6. Program Counter

Feature Additions

1. Adjustable Clock Divider
2. Seven Segment Display

Module Description

1. Program Counter

The program counter module serves as the functional equivalent of the ‘pc’ variable in the RV32I ISA. It stores the location of the instruction within the instruction memory, which is currently being executed. In this implementation, the program counter also handles branching instructions such as BEQ, BLT, BGE and BLTU.

The program counter is also where we have implemented switching between multiple codes. By pressing btnU, the program counter jumps directly to location 528, where the 2nd program begins, while reset takes us back to location 0.

The program counter is clocked, and fetches the next instruction according to the pc_mode variable at every negedge of the clock.

Port Description:

S.No	Port	Type	Width	Description
1	pc	output	32 bits	“pc” variable; The location of the instruction within the instruction memory which is being currently executed.
2	clk	input	1 bit	Clock input.
3	rst	input	1 bit	Reset input.
4	btnU	input	1 bit	Button input, switches to program 2 when pressed.
5	pc_mode	input	2 bits	The mode in which the program counter should be running, one of: ● 00 - Continue to next instruction ● 01 - Branching ● 10 - Move ahead by ‘imm’ instruction memory locations ● 11 - JALR
6	branch_type	input	3 bits	Specify the type of branching, one of: ● 0: BEQ - Branch if Equal ● 1: BNE - Branch if Not Equal ● 2: BLT - Branch if Less Than ● 3: BGE - Branch if Greater Than or Equal ● 4: BLTU - Branch if Less Than, Unsigned ● 5: BGEU - Branch if Greater Than or Equal, Unsigned
7	rs1	input	32 bits	Value of rs1
8	rs2	input	32 bits	Value of rs2
9	imm	input	32 bits	Immediate value passed.

2. Instruction Memory

The instruction memory module holds the instructions to be executed by the processor.The instruction memory we have created can hold up to 1 KiloBytes worth of instructions. At 32 bits per instruction, this gives us a maximum of 256 instructions.

The instruction memory has a somewhat user-friendly interface to input new machine codes, as the user can simply write each instruction into the instruction array, all 32 bits at once. The instruction array will then be rewritten into the actual memory using a for loop, one byte at a time.

In our implementation, we have written two programs in the instruction memory, one which is a bubble sort, and a second which is a basic set which checks all functions of the processor, including reading and writing to data memory, jump statements, and register reading and writing.

Port Description:

S.No	Port	Type	Width	Description
1	writeout	output	32 bits	The 4-byte value in the memory location starting from “loc”, to be written out.
2	loc	input	32 bits	Location of the first byte in the instruction memory from which we load the instruction.

3. Instruction Decode

Now that the PC has fetched the instruction to be executed from the instruction memory, the next step is to decode this instruction according to RV32I’s ISA format in order to determine the type of instruction and the corresponding operation to be performed.
The instructions are broadly classified into six types namely **R**-Register type, **I**-Immediate type,**S**-Store type,**U**-Upper Immediate type, **B**-Branch type and **J**-Jump type each of which is identified by the opcode (first seven bits of the 32-bit instruction). The instructions that share the same opcode are further distinguished using funct3 (3 bits) and funct7 (7 bits) whose positions are defined by RV32I’s ISA.

Port Description:

S.No	Port	Type	Width	Description
1	instruct	input	32 bits	The instruction to be decoded and executed.
2	rs2_or_imm	output	1 bit	If 0 rs2 is the second operand, if 1 immediate value to be used.
3	alu_ctrl	output	4 bits	Determines which arithmetic operation is to be carried out
4	mem_read	output	1 bit	If 1 load from memory, else pass
5	load_type	output	3 bits	Differentiates between load byte(LB),load byte unsigned(LBU),load half word(LH),load half word unsigned(LHU), load word(LW). ● 000 - LB ● 001 - LH ● 010 - LW ● 100 - LBU ● 101 - LHU
6	mem_write	output	1 bit	If 1 write to memory (store), else pass.
7	store_type	output	2 bits	Differentiates between store byte (SB), store half word (SH) and store word (SW). ● 00 - SB ● 01 - SH ● 10 - SW
8	rs1	output	5 bits	Register address of operand 1
9	rs2	output	5 bits	Register address of operand 2
10	imm	output	32 bits	Contains immediate value extracted from instruction and signed extended to 32 bits.
11	pc	output	2 bits	Used to determine the next Program Counter update.. Precisely ● 00 - normal sequential execution ● 01 - Branch statement ● 10 - Jump (JAL) ● 11 - Jump (JALR)
12	branch_type	output	3 bits	Specifies the type of branch operation to be performed. ● 000 - BEQ, Branch if Equal ● 001 - BNE, Branch if Not Equal ● 010 - BLT, Branch if Less Than ● 011 - BGE, Branch if Greater Than or Equal ● 100 - BLTU, Branch if Less Than, Unsigned ● 101 - BGEU, Branch if Greater Than or Equal, Unsigned
13	rd_value	output	2 bits	Selects the source of value to be written in the destination register

S.No	Port	Type	Width	Description
				• 00 - ALU output • 01 - data read from data memory • 10 - pc+4 (return address for jump instruction) • 11 - immediate value
14	reg_write	output	1 bit	If 1 writes to a register, else pass.

4. ALU (Arithmetic Logic Unit)

This module is responsible for the arithmetic and logical operations required during instruction execution.Based on the control signals generated by decode module the ALU module performs operations such as addition, subtraction, comparisons, shift and bitwise operations.

Port description:

S.No	Port	Type	Width	Description
1	A	input	32 bits	Value from rs1
2	B	input	32 bits	Value from either rs2 or sign extended immediate
3	F	input	4 bits	Determines operation to be carried out • 0000 - add • 0001 - subtract • 0010 - bitwise AND • 0011 - bitwise OR • 0100 - bitwise XOR • 0101 - shifts A left by B[4:0] positions • 0110 - shifts A right logically by B[4:0] positions • 0111 - shifts A right arithmetically by B[4:0] positions • 1000 - outputs 1 if A<B (unsigned), else 0 • 1001 - outputs 1 if A<B (signed), else 0
4	C	output	32 bits	Result of the operation

5. Data Memory

The data memory module stores data required during program execution. In this implementation data memory can store up to 32 bytes of data. We can write to data memory using the store instructions and read from it using load instructions.

Port description:

S.No	Port	Type	Width	Description
1	WriteOut	output	32 bits	Data loaded from data memory
2	clk	input	1 bit	Used to synchronise writing data to memory
3	readyn	input	1 bit	Control signal, if 1 load, else pass
4	writeyn	input	1 bit	Control signal, if 1 store, else pass
5	loc	input	32 bits	Specifies memory address for load and store operations
6	load_type	input	3 bits	Specifies load type - load byte(LB),load byte unsigned(LBU),load half word(LH),load half word unsigned(LHU), load word(LW). ● 000 - LB ● 001 - LH ● 010 - LW ● 100 - LBU ● 101 - LHU
7	store type	input	2 bits	Specifies the store type - store byte (SB), store half word (SH) and store word (SW). ● 00 - SB ● 01 - SH ● 10 - SW
8	datain	input	32 bits	Data to be written to memory during store operations

6. Execution

The execution module also serves as the top module in this case. It ties together all the other modules, and serves as an interconnection between them. Also, since the 32 registers need to be accessible by every module within the processor, the registers are also declared within this module, and are passed as arguments to whichever module calls for it.

It is also within this module that we have implemented the ability to view the value of any register on the onboard LEDs. Since there are only 16 LEDs on the board, while each register is 32 bit, we use a separate switch (Switch 6) to switch which half of the word is shown.

All of the 32 registers are initialized to 0 upon reset, and the values for the registers are read and written as per the requirement of each module. Note that the writing takes place at the posedge, while the reading takes place at the negedge of the clock.

Port Description:

S.No	Port	Type	Width	Description
1	led	output	16 bits	The 16 on-board LEDs, used for displaying the value of the register selected using switches 0 (LSB), through 4 (MSB). For Example, in order to see the value of rs19, the switches must be set as: sw[4:0] = {1,0,0,1,1}
2	clk	input	1 bit	Clock input.
3	btnC	input	1 bit	Reset input.
4	btnU	input	1 bit	Button input, switches to program 2 when pressed.
5	sw	input	Array of 16 cells, each 1 bit	On-board switches. They're used as follows: ● 4 to 0: Selecting the register (32 registers = 2^5) ● 6: Whether to display the upper half (ON) or lower half (OFF) ● 13: Hold if ON, Continue if OFF ● 15-14: Select Clock Speed, from 1 Hz (00), 10Hz (01), 100 Hz (10) and full speed (11).
6	seg	output	Array of 7 cells, each 1 bit	Seven Segment Display Cathodes.
7	an	output	Array of 4 cells, each 1 bit	Seven Segment Display Anodes.

7. Clock Divider

The clock divider is a simple counter, which serves to divide the frequency of the on-board clock, to allow us to more clearly see the working of the processor. In our implementation, the clock divider allows for 4 clock speeds, which are 1, 10, 100 Hz and 50 MHz. This is keeping the BASYS-3 board, with an on-board clock of speed 100Mhz in mind.

It works by using a count variable, which counts from 0 to 50,000,000. Every time the count hits 50 Million, the output dips, making one output cycle last 100 Million ticks. The number of ticks per output clock cycle depends on the freqsel variable. If it's set to 1 Hz, then the divider increases by 1 for every on-board clock cycle, needing 100 Million on-board clock cycles to finish one period, which at 100Mhz, takes 1 second.
For 10 Hz, it increases by 10 every on-board clock cycle, and for 100Hz it increases by 100.
At full speed, the output clock switches positions for each period of the on-board clock, effectively dividing the clock speed by 2. Thus, in this implementation, the maximum clock speed is 50 MHz.

The clock divider also supports on-the-fly switching of clock speeds, which is achieved by truncating the count variable as per the frequency selector.

<u>Port Description:</u>

S.No	Port	Type	Width	Description
1	clkout	output	1 bit	Output of the clock divider, switching at the selected frequency.
2	clk	input	1 bit	Clock input.
3	rst	input	1 bit	Reset input.
4	hold	input	1 bit	Only count if Hold is off.
5	freqsel	input	2 bits	Clock Speed Selection. ● 00: 1Hz ● 01: 10 Hz ● 10: 100 Hz ● 11: 50 MHz

8. Seven Segment Display

In this implementation the seven segment display of the Basys 3 FPGA board is used to display the instruction number being executed. We use a total of 138 instructions for the two programs stored in the instruction memory. The instruction number is displayed by multiplexing between four anodes at a sufficiently high frequency to avoid flickering.
This feature proves to be useful in debugging and troubleshooting, as it enables easy tracking of the instruction being executed.

Port description:

S.No	Port	Type	Width	Description
1	seg	output	7 bits	Segment control signals
2	an	output	4 bits	Determines the active anode for digit multiplexing
3	clk	input	1 bit	Used for multiplexing between anodes
4	number	input	8 bits	Instruction number to be displayed Since we are using 8 bits a maximum value of 255 can be displayed (0-255, 256 instuctions) Instruction memory can hold up to a maximum of 256 instructions so this would suffice.

Implemented Programs

Program 1: Bubble Sort Algorithm

A bubble sort algorithm was implemented to validate arithmetic,comparison, and branch operations of the processor. The program repeatedly compares adjacent register values and swaps them if needed. In this implementation, the numbers 45, 12, 78, 3, 50 and 21 are initially loaded into registers 1 to 6 in the given order.Although these six values were chosen for demonstration, the implemented logic can be used for any six values by modifying these initial register values.

Program 2: Basic Functionality Testing

This program simply tests the basic functionality of the processor to ensure accurate execution of the RV32I instructions. This program performs immediate addition (addi), register addition (add), store (SW), load (LW) and branch (BEQ) operations. The result of the arithmetic operation is written to memory, read from memory and compared to validate correct execution of memory access and branching instructions.

Conclusion

A single-cycle RV32I processor was successfully implemented on the Basys 3 FPGA board using Verilog HDL.The implemented processor supports 37 of the 40 RV32I instructions, excluding the FENCE, ECALL and EBREAK. In addition to this, key features such as adjustable clock speed, provision to view register values, provision to switch between programs and provision to view instruction number being executed through the seven segment display of the Basys 3 FPGA made it user- friendly and eased debugging and troubleshooting largely. The project provided valuable insights into CPU architecture and FPGA based implementation.

References

1. “Basic Computer Architecture” - Smruti R Sarangi
2. “Digital Design and Computer Architecture” - David and Sarah Harris
3. “RV32I Base Integer Instruction Set, Version 21” - https://docs.riscv.org/reference/isa/v20240411/unpriv/rv32.html
4. “RISC-V RV32I Base Instruction Set” - https://www.vicilogic.com/static/ext/RISCV/RV32I_BaseInstructionSet.pdf

Link to the Project

You may check out the project here, using the GitHub Repository.

https://github.com/faymere/RV32I-SingleCycleProcessor